

```
In [3]: import numpy as np
import pandas as pd

import IPython.display as dp
import matplotlib.pyplot as plt
import seaborn as sns

dp.set_matplotlib_formats("retina")
sns.set(style="whitegrid", font_scale=1.5)
sns.despine()

%matplotlib inline
```

```
/var/folders/33/j0cl7y453td68qb96j7bqcj4cf41kc/T/ipykernel_45196/321256244
3.py:8: DeprecationWarning: `set_matplotlib_formats` is deprecated since I
Python 7.23, directly use `matplotlib_inline.backend_inline.set_matplotlib
_formats()`
  dp.set_matplotlib_formats("retina")
<Figure size 640x480 with 0 Axes>
```

Lecture

Definitions

$$X_t = R \cos(\omega t + \phi) + Z_t$$

- (Angular) frequency ω
- (Cycle) frequency $f = \omega/2\pi$
- Period or wavelength $T = 1/f = 2\pi/\omega$
- Amplitude R
- Phase ϕ

Spectral representation

$$X_t = R \cos(\omega t + \phi) + Z_t$$

$$X_t = \sum_{j=1}^k R_j \cos(\omega_j t + \phi_j) + Z_t$$

with $\mathbb{E}[R_j] = 0$ or $\phi_j \sim U[0, 2\pi]$

$$X_t = \sum_{j=1}^k (a_j \cos \omega_j t + b_j \sin \omega_j t) + Z_t$$

$$X_t = \int_0^\pi \cos \omega t du(\omega) + \int_0^\pi \sin \omega t dv(\omega)$$

(Power) Spectral Distribution Function

Wiener–Khinchin theorem (spectral representation of ACVF):

$$\gamma(k) = \int_0^\pi \cos \omega k dF(\omega)$$

Properties:

- Monotonically increasing
- $F(0) = 0$
- $F(\pi) = \mathbb{V}\text{ar}(X_t) = \sigma_X^2$

Notes on spectral distribution function

1. Integral limits from 0 to π , because for discrete-time process measured at unit intervals of time:

$$\cos(\omega + k\pi)t = \begin{cases} \cos \omega t, & k \text{ even,} \\ \cos(\pi - \omega)t, & k \text{ odd} \end{cases}$$

Frequency $\omega = \pi$ is called Nyquist frequency.

2. Interpretation: Spectral Distribution Function $F(\omega)$ is the contribution to the variance of the series, which is accounted for by frequencies in the range $(0, \omega)$.
3. If the process contains a deterministic component like $X = R \cos(\omega_0 t + \phi)$, where R is constant and $\phi \sim U[0, 2\pi]$, then $F(\omega)$ is a step function with a step at ω_0 of height

$$\mathbb{V}\text{ar}(X) = \mathbb{E} [R^2 \cos^2(\omega_0 t + \phi)] = \frac{1}{2} R^2$$

4. Define

$$F^*(\omega) = F(\omega)/\sigma_X^2$$

(Power) Spectral Density Function = Spectrum

$$f(\omega) = F'(\omega)$$

$$\gamma(k) = \int_0^\pi \cos \omega k f(\omega) d\omega$$

$$f(\omega) = \frac{1}{\pi} \sum_{k=-\infty}^{\infty} \gamma(k) e^{-i\omega k} = \frac{1}{\pi} \left(\gamma(0) + 2 \sum_{k=1}^{\infty} \gamma(k) \cos \omega k \right)$$

Notes on spectral density function

1. If the process is stationary, the integral will actually not converge. The theorem still holds, but shouldn't be written like that.

2. If we allow frequencies to be negative, we can change their range as follows:

$$\gamma(k) = \int_{-\pi}^{\pi} e^{i\omega k} f(\omega) d\omega$$

$$f(\omega) = \frac{1}{2\pi} \sum_{k=-\infty}^{\infty} \gamma(k) e^{-i\omega k}$$

3. If we normalize similarly to spectral distribution function, ACF will appear:

$$f^*(\omega) = f(\omega)/\sigma_X^2 = \frac{1}{\pi} \left(1 + 2 \sum_{k=1}^{\infty} \rho(k) \cos \omega k \right)$$

4. For a continuous purely indeterministic stationary process $X(t)$:

$$f(\omega) = \frac{1}{\pi} \int_{-\infty}^{\infty} \gamma(\tau) e^{-i\omega\tau} d\tau = \frac{2}{\pi} \int_0^{\infty} \gamma(\tau) e^{-i\omega\tau} d\tau$$

$$\gamma(\tau) = \int_0^{\infty} f(\omega) \cos \omega\tau d\omega$$

Example 1: spectrum of white noise

For white noise Z_t with $\mathbb{V}\text{ar}(Z_t) = \sigma_Z^2$, using

$$f(\omega) = \frac{1}{\pi} \left(\gamma(0) + 2 \sum_{k=1}^{\infty} \gamma(k) \cos \omega k \right) = \gamma(0)/\pi = \sigma_Z^2/\pi$$

Example 2: spectrum of MA(1)

Consider $X_t = Z_t + \beta Z_{t-1}$. We know its ACF:

$$\rho(k) = \begin{cases} 1, & k = 0, \\ \beta/(1 + \beta^2), & k = \pm 1, \\ 0, & \text{else} \end{cases}$$

Using

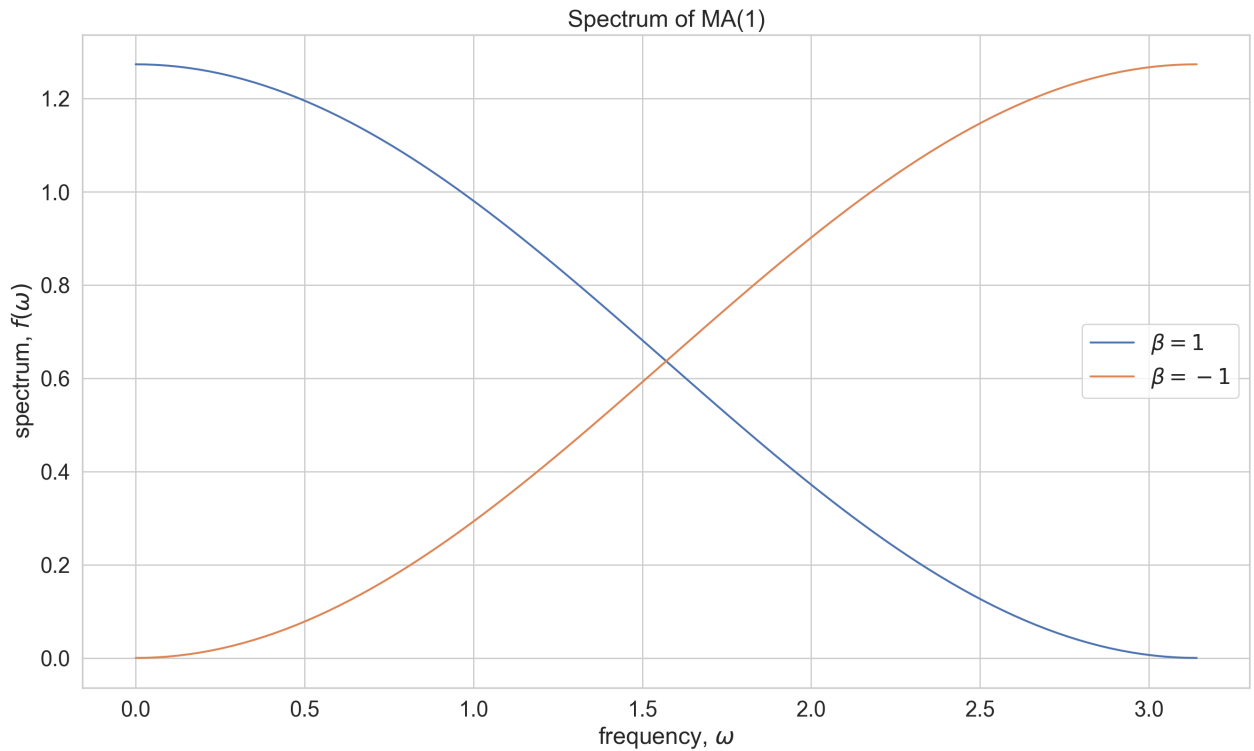
$$f^*(\omega) = \frac{1}{\pi} \left(1 + 2 \sum_{k=1}^{\infty} \rho(k) \cos \omega k \right) = \frac{1}{\pi} \left(1 + \frac{2\beta}{1 + \beta^2} \cos \omega \right)$$

Then,

$$f(\omega) = \sigma_X^2 f^*(\omega) = (1 + \beta^2) \sigma_Z^2 f^*(\omega)$$

```
In [15]: w = np.linspace(0, np.pi, 100)
sigma_z = 1
beta = 1
f_pos = (1 + beta ** 2) * sigma_z ** 2 * (1 + 2 * beta * np.cos(w) / (1 +
f_neg = (1 + beta ** 2) * sigma_z ** 2 * (1 - 2 * beta * np.cos(w) / (1 +
```

```
In [16]: fig, ax = plt.subplots(figsize=(16,9))
ax.plot(w, f_pos, label="$\\beta = 1$")
ax.plot(w, f_neg, label="$\\beta = -1$")
ax.set_xlabel("frequency, $\\omega$")
ax.set_ylabel("spectrum, $f(\\omega)$")
ax.set_title("Spectrum of MA(1)")
ax.legend();
```



Example 3: spectrum of AR(1)

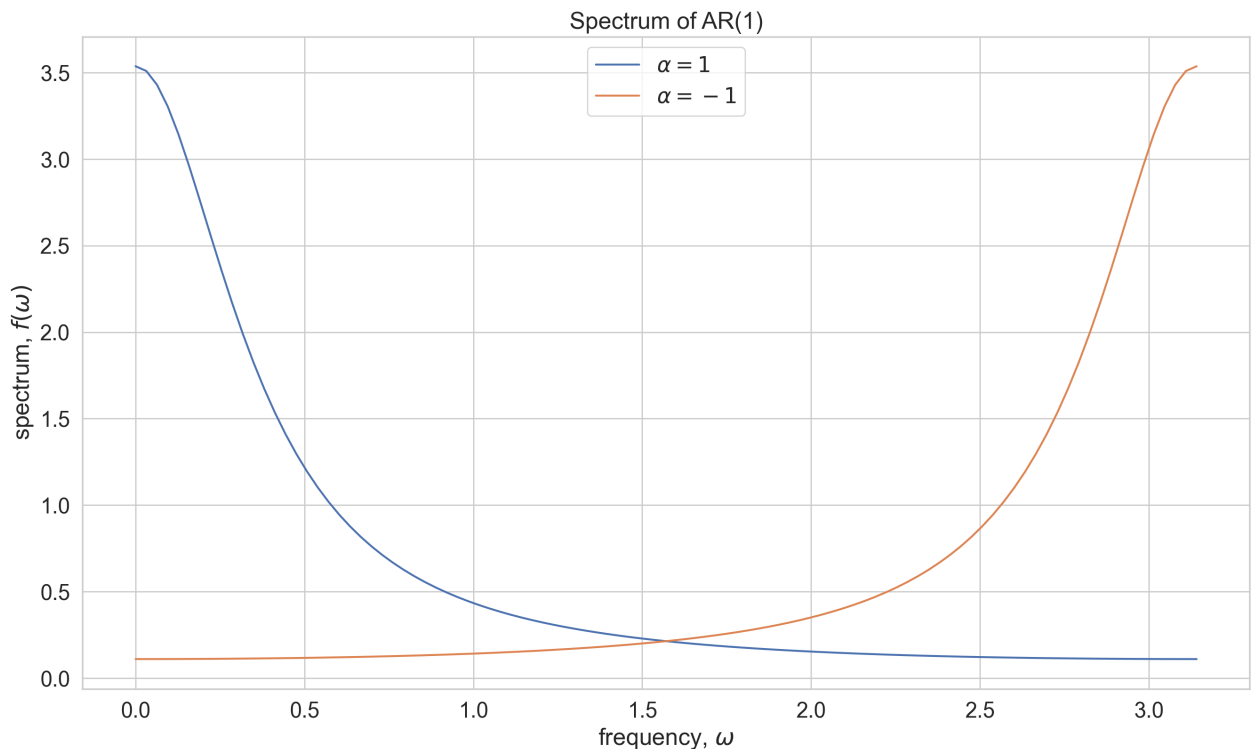
Consider $X_t = \alpha X_{t-1} + Z_t$. We know its ACVF $\gamma(k) = \sigma_X^2 \alpha^{|k|}$. Using

$$\begin{aligned}
 f(\omega) &= \frac{1}{\pi} \sum_{k=-\infty}^{\infty} \gamma(k) e^{-i\omega k} = \\
 &= \frac{\sigma_X^2}{\pi} \left(1 + \sum_{k=1}^{\infty} \alpha^k e^{-i\omega k} + \sum_{k=1}^{\infty} \alpha^k e^{i\omega k} \right) = \\
 &= \frac{\sigma_X^2}{\pi} \left(1 + \frac{\alpha e^{-i\omega}}{1 - \alpha e^{-i\omega}} + \frac{\alpha e^{i\omega}}{1 - \alpha e^{i\omega}} \right) = \\
 &= \frac{\sigma_X^2}{\pi} \frac{1 - \alpha^2}{1 - 2\alpha \cos \omega + \alpha^2} = \frac{\sigma_Z^2}{\pi(1 - 2\alpha \cos \omega + \alpha^2)}
 \end{aligned}$$

```
In [44]: w = np.linspace(0, np.pi, 100)
sigma_z = 1
alpha = 0.7
f_pos = sigma_z ** 2 / (np.pi * (1 - 2 * alpha * np.cos(w) + alpha ** 2))
f_neg = sigma_z ** 2 / (np.pi * (1 + 2 * alpha * np.cos(w) + alpha ** 2))
```

```
In [45]: fig, ax = plt.subplots(figsize=(16,9))
ax.plot(w, f_pos, label="$\\alpha = 1$")
ax.plot(w, f_neg, label="$\\alpha = -1$")
ax.set_xlabel("frequency, $\\omega$")
ax.set_ylabel("spectrum, $f(\\omega)$")
ax.set_title("Spectrum of AR(1)")
```

```
ax.legend();
```



Example 4: spectrum of AR(2)

Consider $X_t = \alpha_1 X_{t-1} + \alpha_2 X_{t-2} + Z_t$.

Without derivation,

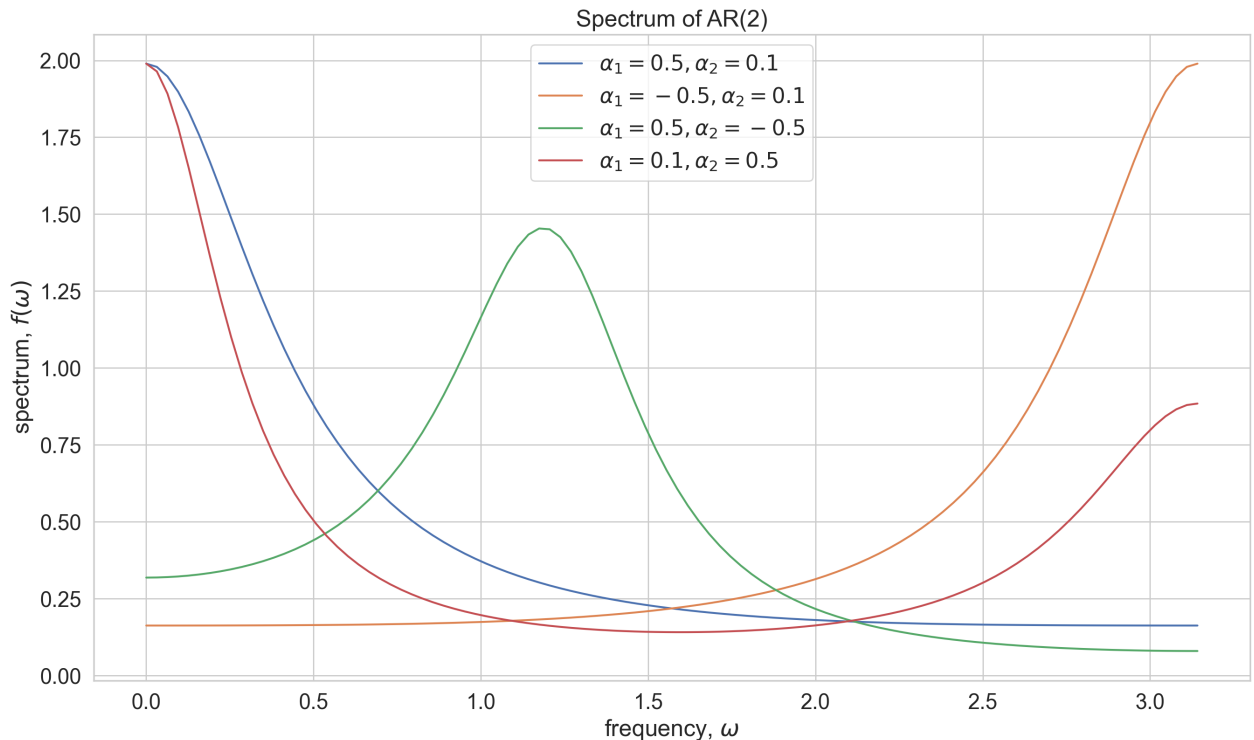
$$f(\omega) = \frac{\sigma_Z^2}{\pi (1 + \alpha_1^2 + \alpha_2^2 - 2\alpha_1(1 - \alpha_2) \cos \omega - 2\alpha_2 \cos 2\omega)}$$

```
In [36]: def ar_2_f(w, sigma_z, alpha_1, alpha_2):
         return sigma_z ** 2 / (np.pi * (1 + alpha_1 ** 2 + alpha_2 ** 2 - 2 * alpha_1 * (1 - alpha_2) * np.cos(w) - 2 * alpha_2 * np.cos(2 * w)))
```

```
In [46]: w = np.linspace(0, np.pi, 100)
         sigma_z = 1
         f_left = ar_2_f(w, sigma_z, 0.5, 0.1)
         f_mid = ar_2_f(w, sigma_z, 0.5, -0.5)
         f_right = ar_2_f(w, sigma_z, -0.5, 0.1)
         f_mid_low = ar_2_f(w, sigma_z, 0.1, 0.5)
```

```
In [47]: fig, ax = plt.subplots(figsize=(16,9))
         ax.plot(w, f_left, label="$\\alpha_1 = 0.5, \\alpha_2 = 0.1$")
         ax.plot(w, f_right, label="$\\alpha_1 = -0.5, \\alpha_2 = 0.1$")
         ax.plot(w, f_mid, label="$\\alpha_1 = 0.5, \\alpha_2 = -0.5$")
         ax.plot(w, f_mid_low, label="$\\alpha_1 = 0.1, \\alpha_2 = 0.5$")
         ax.set_xlabel("frequency, $\\omega$")
         ax.set_ylabel("spectrum, $f(\\omega)$")
```

```
ax.set_title("Spectrum of AR(2)")
ax.legend();
```



Example 5: spectrum of deterministic process

Consider $X_t = \cos(\omega_0 t + \phi)$, where $\phi \sim U[0, 2\pi]$. As discussed previously,

$$\gamma(k) = \frac{1}{2} \cos \omega_0 k$$

$$F(\omega) = \begin{cases} 0, & \omega < \omega_0, \\ \frac{1}{2}, & \omega \geq \omega_0 \end{cases}$$

$$f(\omega) = 0, \quad \forall \omega \neq \omega_0$$

Spectral analysis

Spectral analysis is the name given to methods of estimating the spectral density function, or spectrum, of a given time series. Traditional spectral analysis is essentially a modification of Fourier analysis so as to make it suitable for stochastic rather than deterministic functions of time. The Fourier series representation has N parameters to describe N observations and so can be made to fit the data exactly. The overall effect of the Fourier analysis of the data is to partition the variability of the series into components at the specified N frequencies called harmonics.

Discrete Fourier Transform

Given time series X_t , DFT transforms it into the same number of frequency components ω_k :

$$\omega_k = \sum_{t=0}^{T-1} X_t e^{-2\pi i k t / T}$$

Parseval's theorem

The Fourier transform is **unitary**. Loosely, that the sum of the square of a function is equal to the sum of the square of its transform:

$$\sum_{t=0}^{T-1} |X_t|^2 = \frac{1}{T} \sum_{k=0}^{T-1} |\omega_k|^2 = \sum_{k=0}^{T-1} \frac{1}{T} |\omega_k|^2 = \sum_{k=0}^{T-1} I(\omega_k)$$

Periodogram

$$I(\omega) = \frac{1}{T} |\omega|^2$$

Plot $I(\omega)$ vs ω is called periodogram.

Properties of periodogram

1. Periodogram is the DFT of ACVF:

$$I(\omega_p) = \frac{1}{\pi} \left(c_0 + 2 \sum_{k=1}^{T-1} c_k \cos \omega_p k \right)$$

2. Asymptotically unbiased estimate of the power spectrum
3. Not consistent estimate!

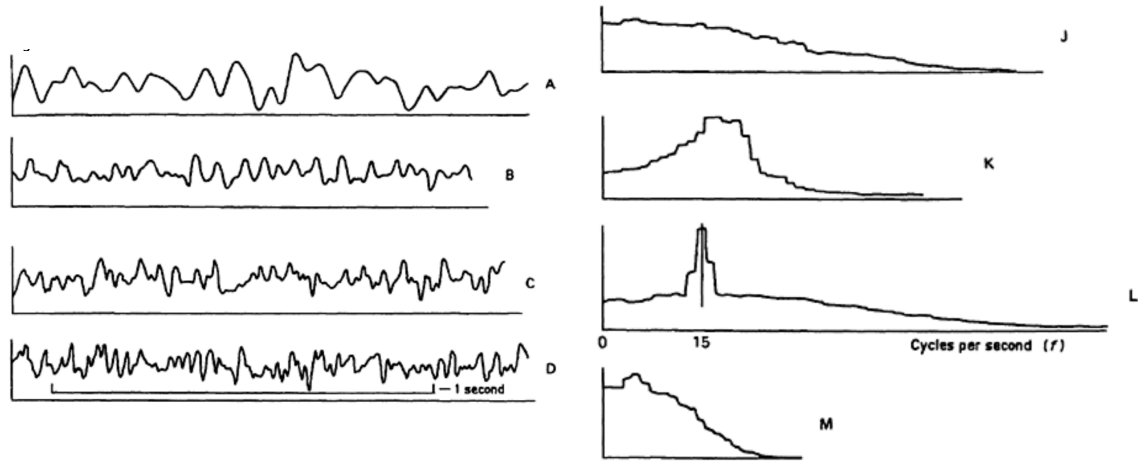
Better estimates

$$\hat{f}(\omega) = \frac{1}{\pi} \left(\lambda_0 c_0 + 2 \sum_{k=1}^M \lambda_k c_k \cos \omega k \right)$$

1. Tukey window

$$\lambda_k = \frac{1}{2} \left(1 + \cos \frac{\pi k}{M} \right)$$

Example



Webinar

```
In [51]: import numpy as np
import pandas as pd

import IPython.display as dp
import matplotlib.pyplot as plt
import seaborn as sns

dp.set_matplotlib_formats("retina")
sns.set(style="whitegrid", font_scale=1.5)
sns.despine()

%matplotlib inline
```

```
/var/folders/33/j0cl7y453td68qb96j7bqcj4cf41kc/T/ipykernel_45196/321256244
3.py:8: DeprecationWarning: `set_matplotlib_formats` is deprecated since I
Python 7.23, directly use `matplotlib_inline.backend_inline.set_matplotlib
_formats()`
  dp.set_matplotlib_formats("retina")
<Figure size 640x480 with 0 Axes>
```

```
In [52]: from darts import TimeSeries
import darts.datasets as ds
import scipy.stats as sts
import statsmodels.api as sm
import statsmodels.tsa.api as tsa
from darts.utils.statistics import plot_acf
```

```
In [48]: from scipy.signal import periodogram
```

```
In [60]: from scipy.signal import find_peaks
```

```
In [49]: from astsadata import speech, cmort
```

```
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadata/astsadata.py:15: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.
  birth = pd.read_csv(f"{path}/data/birth.csv", index_col=0, parse_dates=True)
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadata/astsadata.py:27: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.
  cardox = pd.read_csv(f"{path}/data/cardox.csv", index_col=0, parse_dates=True)
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadata/astsadata.py:30: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.
  chicken = pd.read_csv(f"{path}/data/chicken.csv", index_col=0, parse_dates=True)
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadata/astsadata.py:36: UserWarning: Converting to PeriodArray/Index representation will drop timezone information.
  cmort.index = cmort.index.to_period("W")
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadata/astsadata.py:54: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.
  flu = pd.read_csv(f"{path}/data/flu.csv", index_col=0, parse_dates=True)
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadata/astsadata.py:61: UserWarning: Converting to PeriodArray/Index representation will drop timezone information.
  gas.index = gas.index.to_period("W")
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadata/astsadata.py:63: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.
  gdp = pd.read_csv(f"{path}/data/gdp.csv", index_col=0, parse_dates=True)
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadata/astsadata.py:72: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.
  gnp = pd.read_csv(f"{path}/data/gnp.csv", index_col=0, parse_dates=True)
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadata/astsadata.py:90: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.
  hor = pd.read_csv(f"{path}/data/hor.csv", index_col=0, parse_dates=True)
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadata
```

```
a/astsadata.py:93: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.
  jj = pd.read_csv(f"{path}/data/jj.csv", index_col=0, parse_dates=True)
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadata/astsadata.py:97: UserWarning: Converting to PeriodArray/Index representation will drop timezone information.
  lap.index = lap.index.to_period("W")
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadata/astsadata.py:109: UserWarning: Converting to PeriodArray/Index representation will drop timezone information.
  oil.index = oil.index.to_period("W")
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadata/astsadata.py:112: UserWarning: Converting to PeriodArray/Index representation will drop timezone information.
  part.index = part.index.to_period("W")
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadata/astsadata.py:117: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.
  polio = pd.read_csv(f"{path}/data/polio.csv", index_col=0, parse_dates=True)
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadata/astsadata.py:120: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.
  prodn = pd.read_csv(f"{path}/data/prodn.csv", index_col=0, parse_dates=True)
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadata/astsadata.py:123: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.
  qinfl = pd.read_csv(f"{path}/data/qinfl.csv", index_col=0, parse_dates=True)
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadata/astsadata.py:126: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.
  qintr = pd.read_csv(f"{path}/data/qintr.csv", index_col=0, parse_dates=True)
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadata/astsadata.py:129: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.
  rec = pd.read_csv(f"{path}/data/rec.csv", index_col=0, parse_dates=True)
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadata/astsadata.py:135: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.
  salmon = pd.read_csv(f"{path}/data/salmon.csv", index_col=0, parse_dates=True)
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadata/astsadata.py:145: UserWarning: Converting to PeriodArray/Index representation will drop timezone information.
```

```

so2.index = so2.index.to_period("W")
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadata/astsadata.py:147: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.
soi = pd.read_csv(f"{path}/data/soi.csv", index_col=0, parse_dates=True)
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadata/astsadata.py:158: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.
sunspotz = pd.read_csv(f"{path}/data/sunspotz.csv", index_col=0, parse_dates=True)
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadata/astsadata.py:162: UserWarning: Converting to PeriodArray/Index representation will drop timezone information.
tempr.index = tempr.index.to_period("W")
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadata/astsadata.py:164: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.
unemp = pd.read_csv(f"{path}/data/unemp.csv", index_col=0, parse_dates=True)
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadata/astsadata.py:167: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.
UnempRate = pd.read_csv(f"{path}/data/UnempRate.csv", index_col=0, parse_dates=True)
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadata/astsadata.py:176: FutureWarning: The argument 'date_parser' is deprecated and will be removed in a future version. Please use 'date_format' instead, or read your data in as 'object' dtype and then call 'to_datetime'.
djia = pd.read_csv(
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadata/astsadata.py:180: FutureWarning: The behavior of 'to_datetime' with 'unit' when parsing strings is deprecated. In a future version, strings will be parsed as datetime strings, matching the behavior without a 'unit'. To retain the old behavior, explicitly cast ints or floats to numeric type before calling to_datetime.
date_parser=lambda x: pd.to_datetime(x, unit="s"),
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadata/astsadata.py:182: FutureWarning: PeriodDtype[B] is deprecated and will be removed in a future version. Use a DatetimeIndex with freq='B' instead
djia.index = djia.index.to_period("B")

```

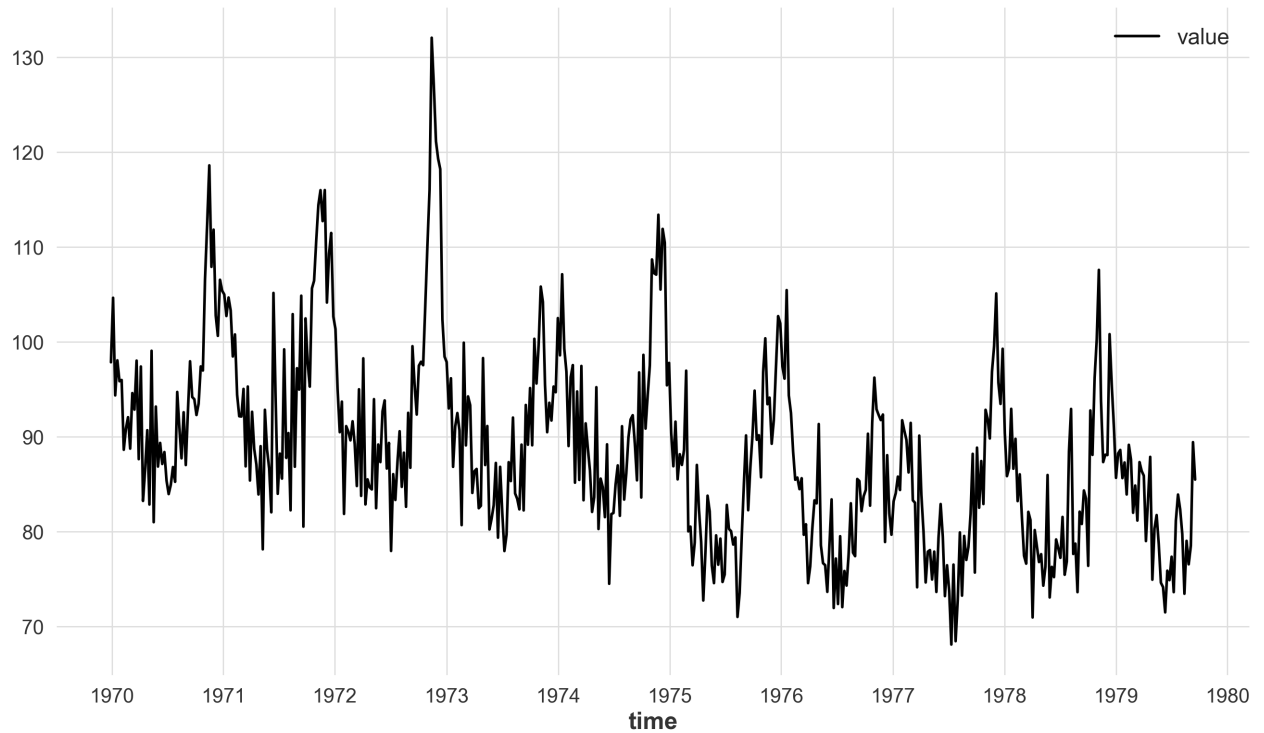
```

In [53]: cmort_ts = TimeSeries.from_series(cmort.set_index(
    pd.date_range(
        cmort.index[0].start_time.date(),
        periods=508,
        freq="W-MON",
        inclusive="both"
    )
))

```

```
In [73]: fig, ax = plt.subplots(figsize=(16,9))  
cmort_ts.plot(ax=ax)
```

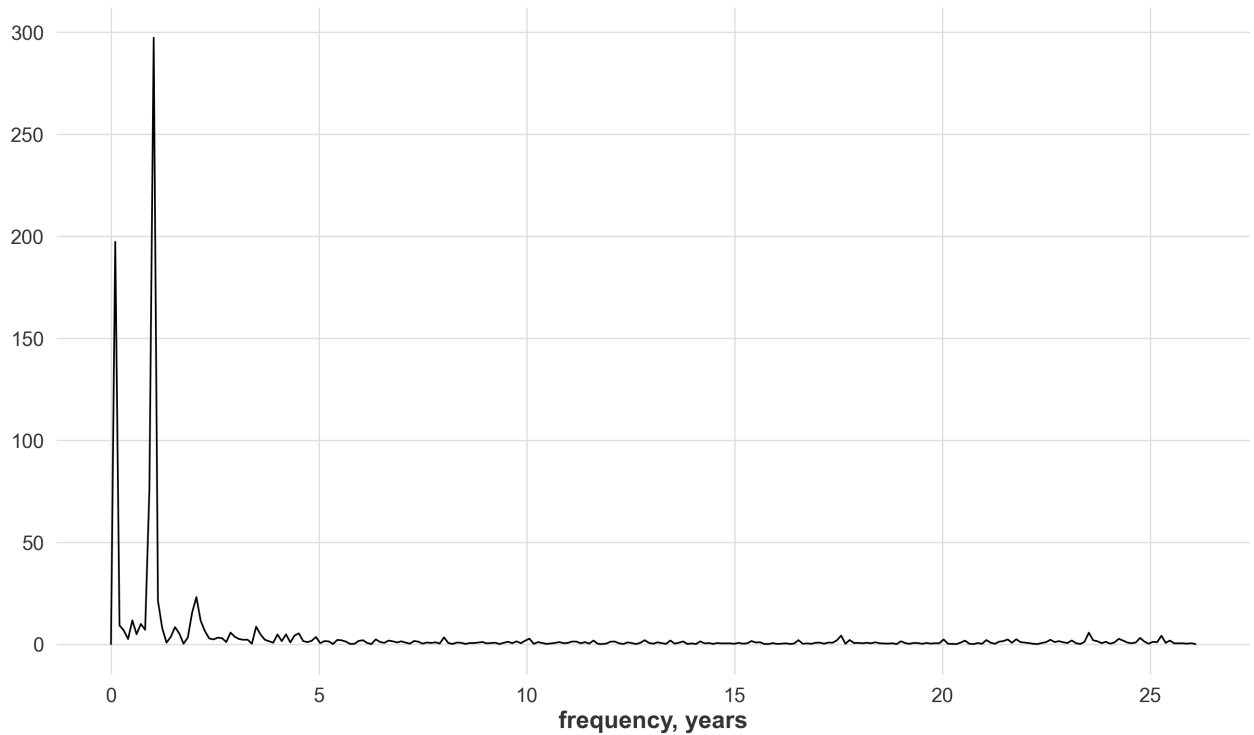
```
Out[73]: <Axes: xlabel='time'>
```



```
In [69]: freq, psd = periodogram(cmort_ts.values().flatten(), fs=52.18)
```

```
In [85]: fig, ax = plt.subplots(figsize=(16,9))  
ax.plot(freq, psd)  
ax.set_xlabel("frequency, years")
```

```
Out[85]: Text(0.5, 0, 'frequency, years')
```

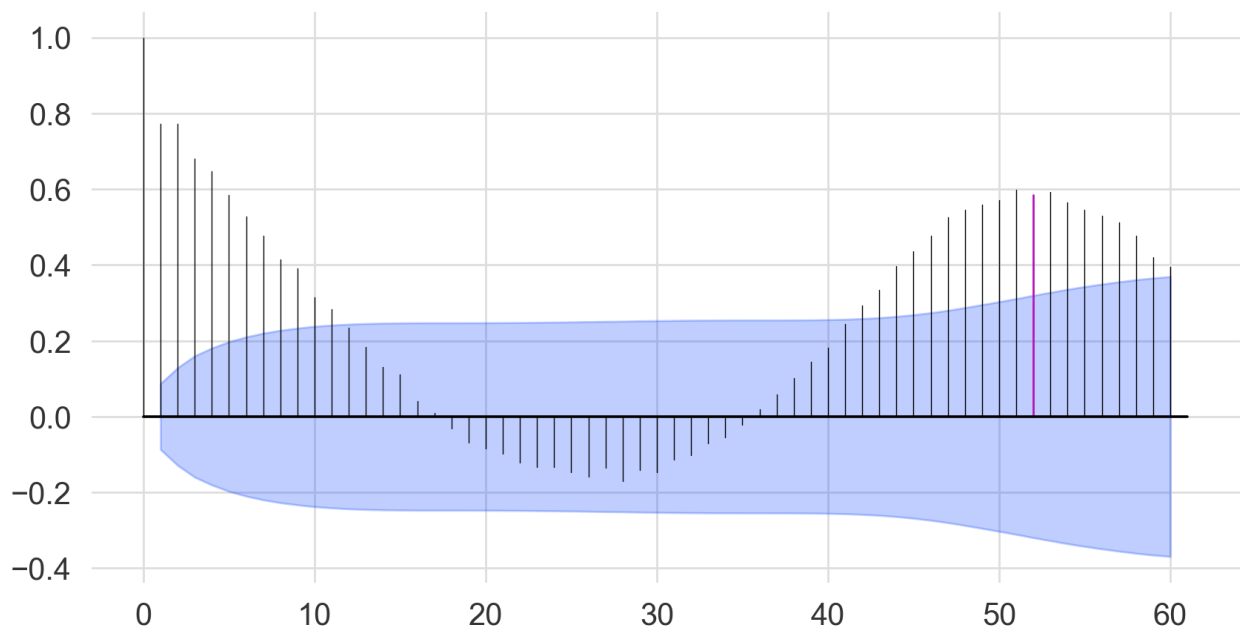


```
In [87]: freq[find_peaks(psd, threshold=50)[0]]
```

```
Out[87]: array([0.10271654, 1.02716535])
```

```
In [124]: from darts.utils.statistics import plot_acf
```

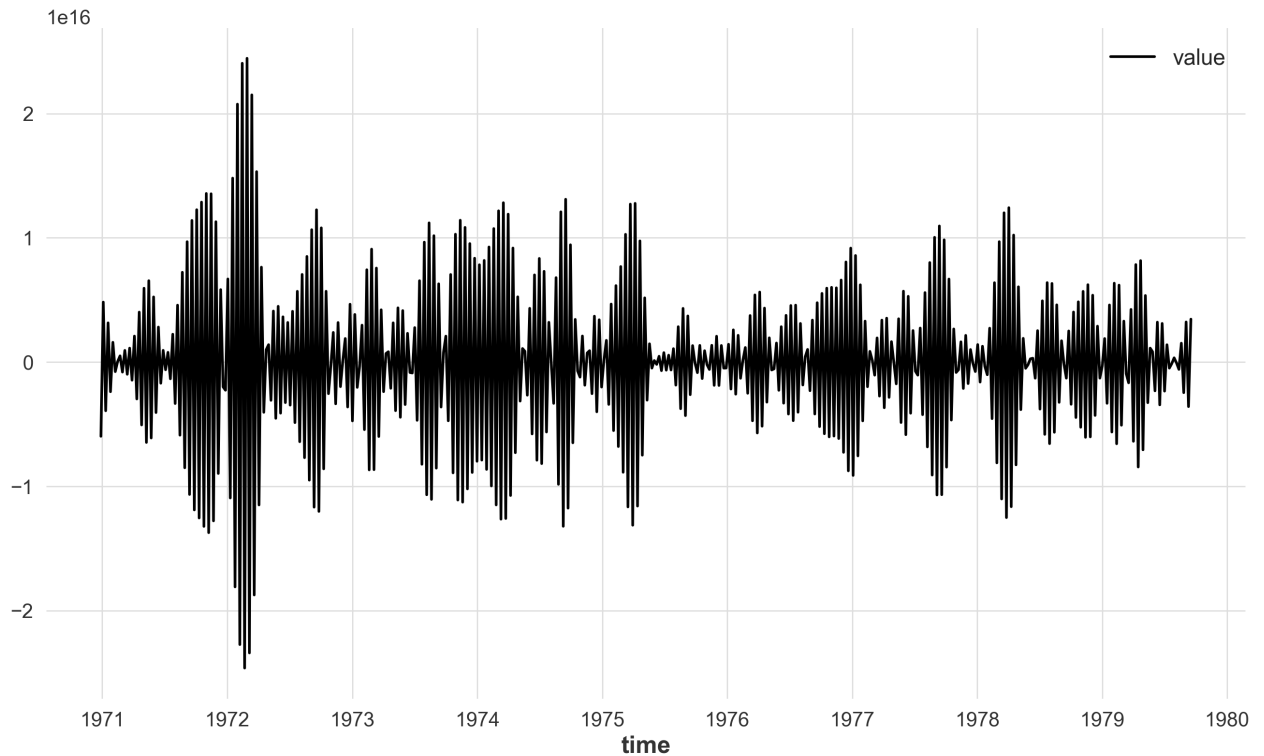
```
In [126]: plot_acf(cmort_ts, max_lag=60, m=52)
```



```
In [88]: cmort_sdiff_ts = cmort_ts.diff(52)
```

```
In [89]: fig, ax = plt.subplots(figsize=(16,9))
cmort_sdiff_ts.plot(ax=ax)
```

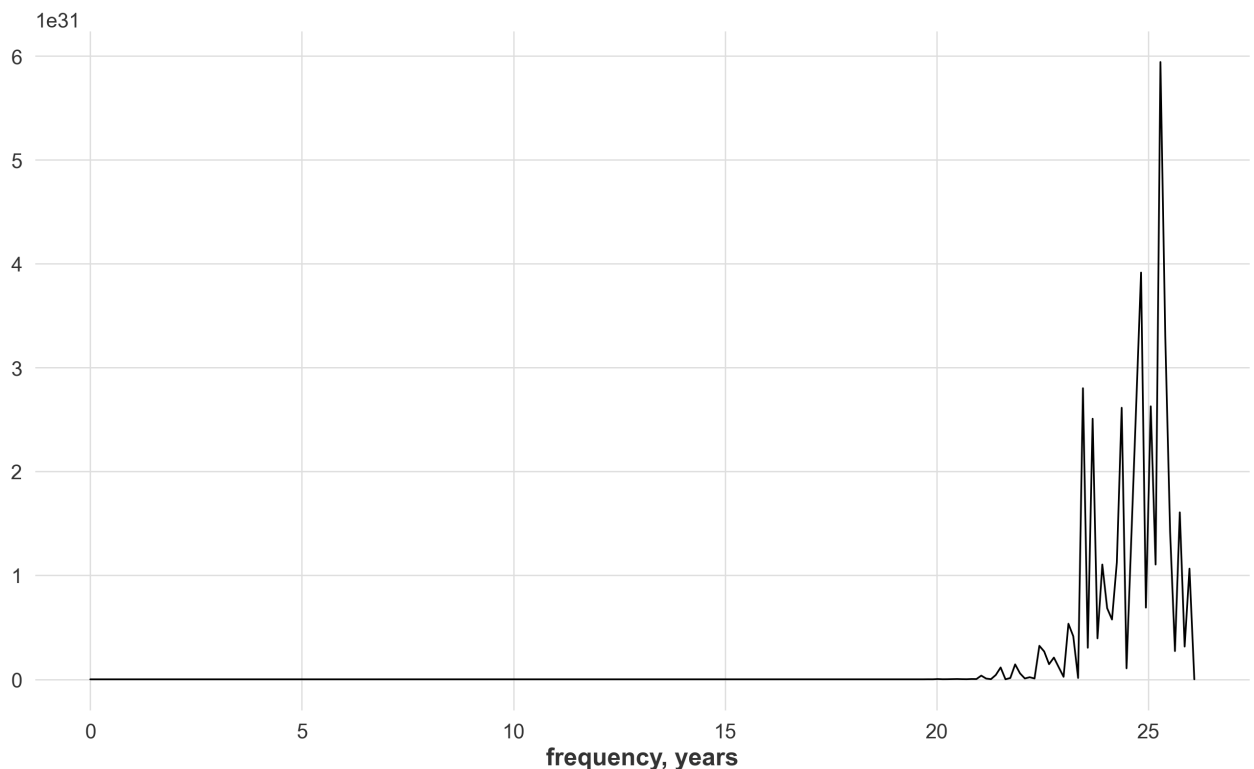
Out[89]: <Axes: xlabel='time'>



```
In [129... freq2, psd2 = periodogram(cmort_sdiff_ts.values().flatten(), fs=52.18)
```

```
In [130... fig, ax = plt.subplots(figsize=(16,9))
ax.plot(freq2, psd2)
ax.set_xlabel("frequency, years")
```

Out[130... Text(0.5, 0, 'frequency, years')



In [131... 52/21

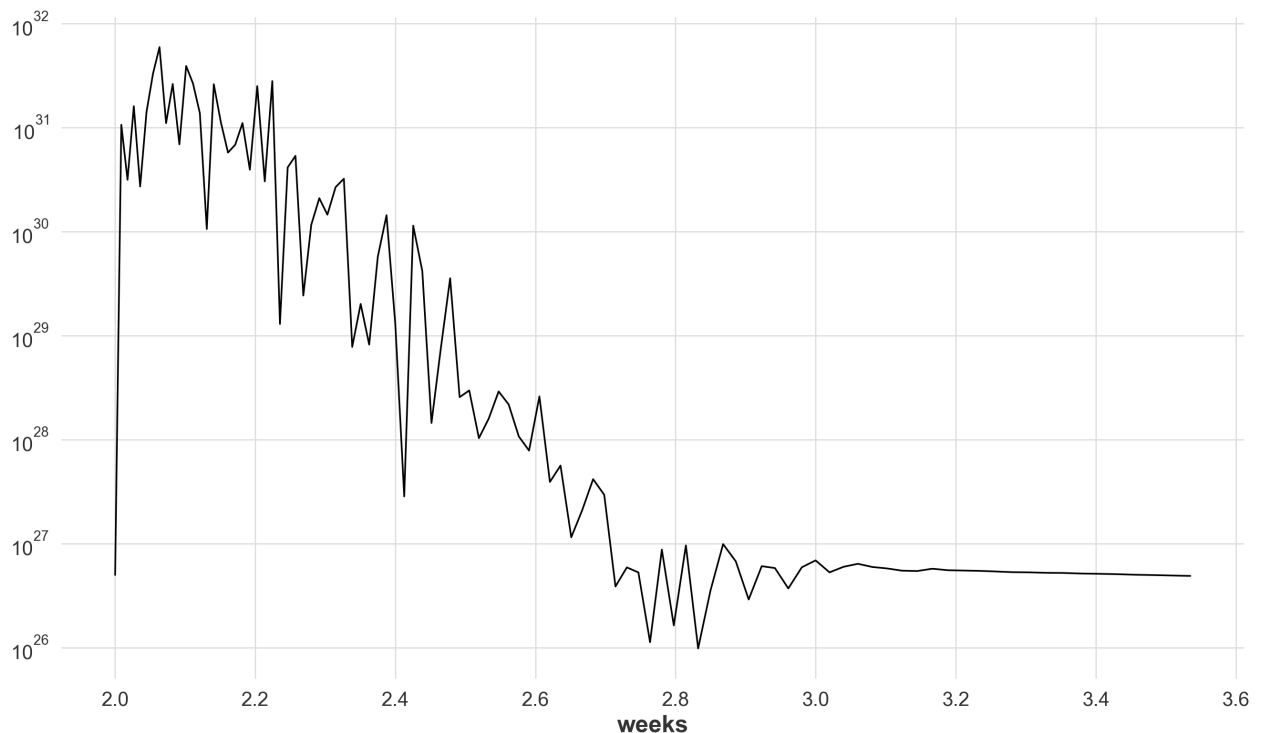
Out[131... 2.4761904761904763

In [132... 52/26

Out[132... 2.0

```
In [133... fig, ax = plt.subplots(figsize=(16,9))
ax.semilogy(52.18/freq2[-100:], psd2[-100:])
ax.set_xlabel("weeks")
```

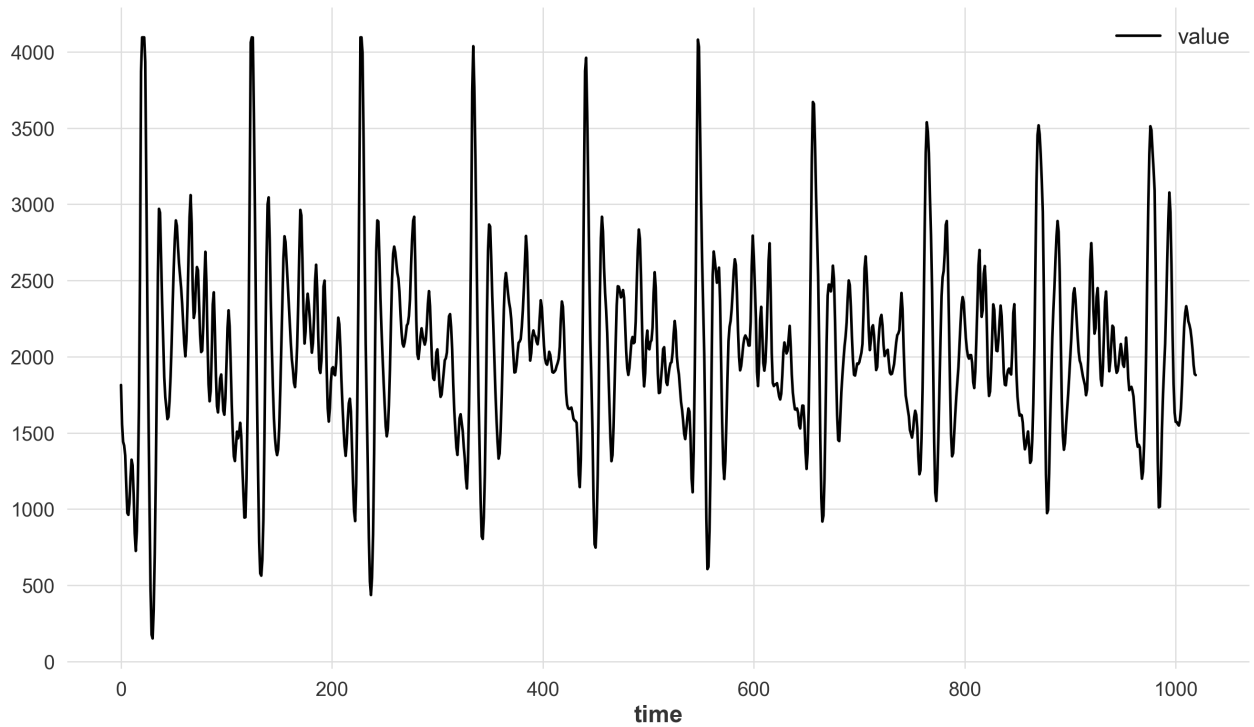
Out[133... Text(0.5, 0, 'weeks')



```
In [113... speech_ts = TimeSeries.from_series(speech.reset_index(drop=True))
```

```
In [114... fig, ax = plt.subplots(figsize=(16,9))
speech_ts.plot(ax=ax)
```

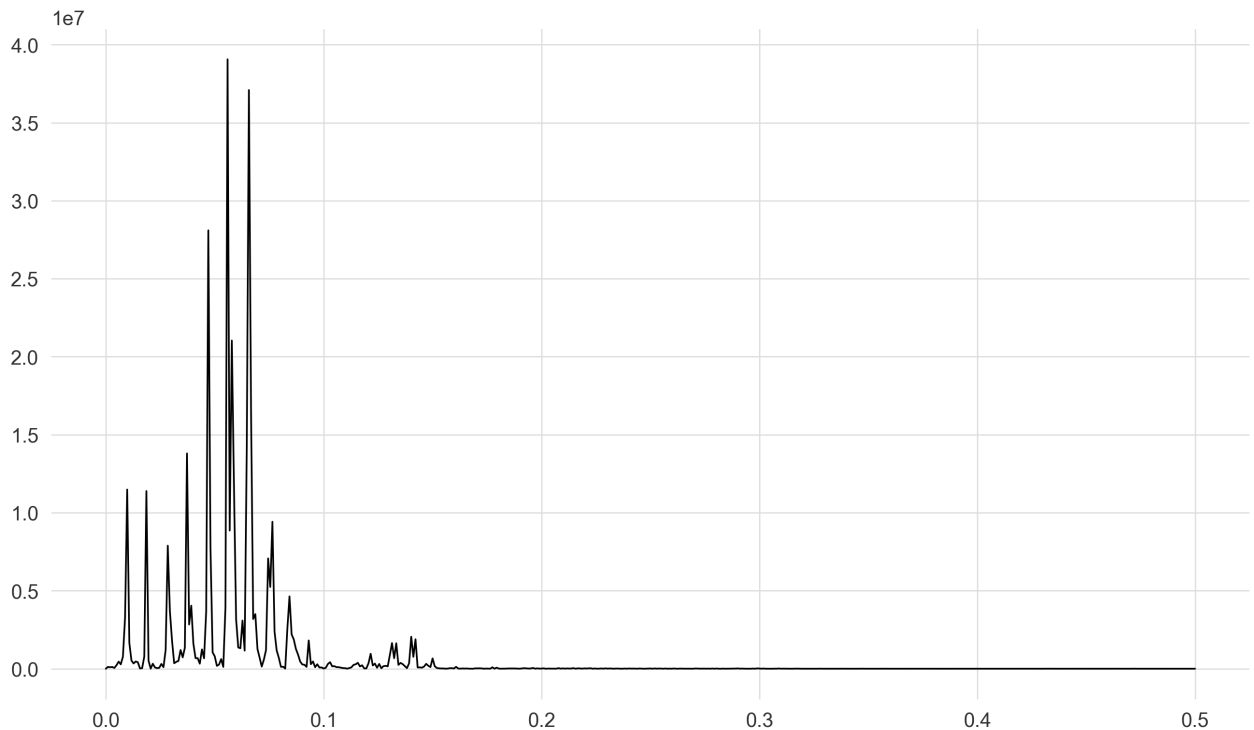
Out[114... <Axes: xlabel='time'>



```
In [116... freq3, psd3 = periodogram(speech_ts.values().flatten())
```

```
In [120... fig, ax = plt.subplots(figsize=(16,9))
ax.plot(freq3, psd3)
```

```
Out[120... [<matplotlib.lines.Line2D at 0x2ad6d9f70>]
```



```
In [118... freq3[find_peaks(psd3, threshold=5e6)[0]]
```

```
Out[118... array([0.00980392, 0.01862745, 0.0372549 , 0.04705882, 0.05588235,
0.05784314, 0.06568627])
```

```
In [119... 1/freq3[find_peaks(psd3, threshold=5e6)[0]]
```

```
Out[119... array([102.          , 53.68421053, 26.84210526, 21.25          ,  
                17.89473684, 17.28813559, 15.2238806 ])
```